

Replication and Fault-Tolerance

1. Why It Is Needed

In aggregation pipelines, if a host crashes:

None

All in-memory aggregated counts are lost

Example:

None

A=500

B=900

C=300

If host fails before flush → data lost.

2. Cascading Failure Problem

If one deeper-level host fails:

None

Upstream hosts cannot forward data

Their memory fills up

They overflow

More data lost

Pipeline delay increases

One failure can affect many levels.

3. Why Simple Recovery Is Costly

Possible recovery methods:

None

Checkpointing

Dead Letter Queue

Replay events

Reprocessing

But if large tiers fail:

None

Huge recomputation needed

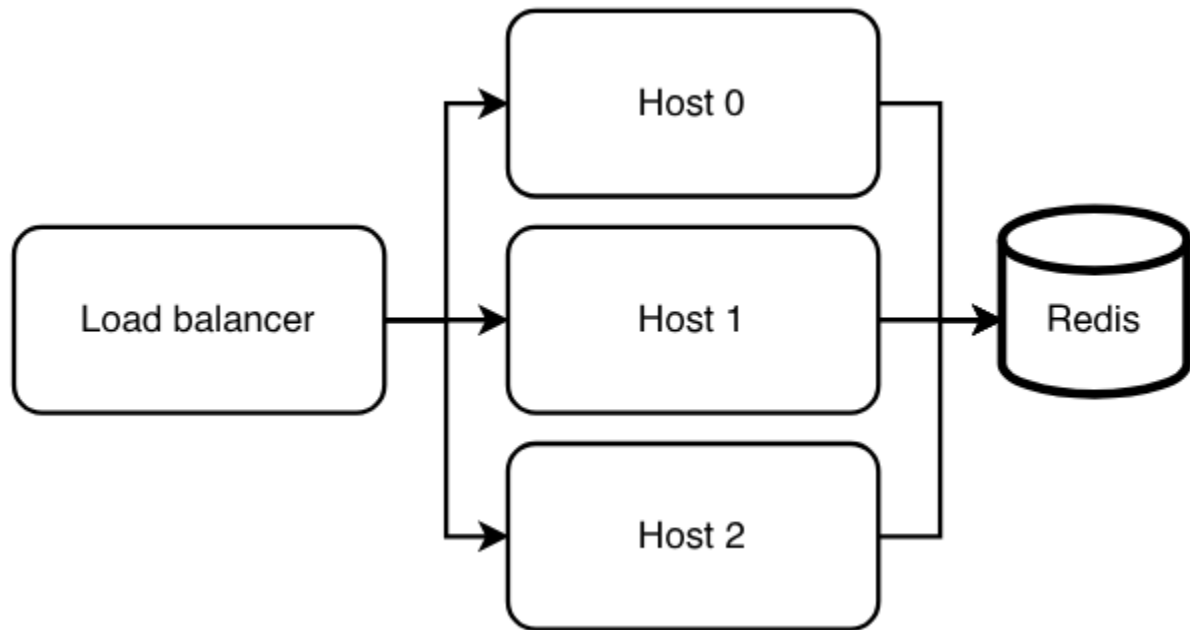
Extra CPU cost

More latency

Waste of resources

4. Better Solution

Convert each node into an **independent aggregation service**.



Each aggregation unit contains:

None

Load Balancer

Host 0

Host 1

Host 2

Shared Redis

As shown in Figure.

5. How It Works

Incoming requests:

None

Load Balancer



Any available stateless host



Update counters in Redis

Since hosts are stateless:

None

If one host dies,

other hosts continue processing

Data remains safe in Redis.

6. Why Stateless Hosts Help

Hosts do not keep aggregation data locally.

They only process requests.

Benefits:

None

Easy replacement

Auto-scaling

No data loss on host crash

Fast failover

7. Why Redis Is Used

Redis provides:

None

In-memory speed

Shared state

Atomic increments

Low latency

Replication support

Good for counters.

8. Does This Increase Writes?

Yes, but manageable.

Because:

None

Each service has separate Redis cluster

No contention on same keys

Temporary data only

Deleted after flush

So storage growth stays controlled.

9. Example Flow

None

Host 0 receives A event

Redis A=10

Host 1 receives A event

Redis A=11

Host 2 receives B event

Redis B=7

Later flushed to database.

10. Fault-Tolerance Level

Even if one host fails:

None

3 hosts total

1 down

2 still running

Service continues

Can use more replicas if needed.

11. Modern Deployment Example

Use:

None

Terraform

Kubernetes

3 Pods

Redis Cluster

Load Balancer

Fully cloud-native architecture.

12. Advantages

None

No in-memory count loss

High availability

Easy scaling

Fast recovery

Reliable aggregation

13. Disadvantages

None

Extra infrastructure

Redis cost

Operational complexity

Network dependency

14. Interview Talking Point

If asked how to avoid aggregation node failures:

None

Replace node with stateless service

Use 3 hosts + load balancer + Redis

Persist shared counters

Hosts can fail safely

Strong HLD answer.

15. One-Line Summary

For fault-tolerance, replace each aggregation node with multiple stateless hosts using a shared Redis store, so host failures do not lose aggregated data and service remains available.